Matching model: Model Development Guidelines check (update)

For the matching team (MAUDE ↔ GUDID same-device model) · 13 November 2026 · follows the check of 12 November on `model-dev-integration` at commit 428163f · work on branch `model-dev/guidelines-ci`

Summary

Every item the 12 November check listed as Missing or Partial now has code behind it, except the choice between the four model variants, which is still made by hand. The matching model has a **model gate** (metric floors, no worse than the champion, bias gaps between slices) that fails CI and keeps the champion when a new model is worse; s11 **publishes** the gated model to Artifact Registry by itself; and there is a **training Docker image**, a **model comparison chart** and a **hyperparameter sensitivity** analysis. Bias mitigation with the pipeline's IPW weights was tried and changed nothing (documented). The gate was proven on real runs: it passes the released model and **blocks** a 20-trial retrain that is worse on the hand sheet. The cause of the `contains` false positives was found (a label rule disagrees with the hand guide) and fixed with a serving gate, released as **v1.1.0**: hand precision 0.959 → 1.000, no false positives, and the bias gap between slices is gone. One thing is left: the GCP variables and roles, so CI can train and publish by itself. Hand batch 2 was taken off the task list, so the hand sheet stays the only hand set and the hand numbers are optimistic.

1. What was checked

Check	Result
Matching tests (pytest, Python 3.12 venv)	538 passed (509 before + 29 new: gate, publish, IPW, workflow, IE gate versions)
Smoke chain, the CI steps (fs2 table + TF-IDF encoder, s05, s06 with 1 trial, check_training_outputs.py, s08 for every model, gate --report-only)	Passes end to end; the gate runs and reports (synthetic numbers, so it is report-only there)
Docker image `vigil-matching-train`	Builds; on the smoke data it gives exactly the same numbers as the scripts outside the container
Gate: v1.0.0 against itself	Passes (exit 0), one warning: precision gap between slices 0.158 vs a 0.15 ceiling (see 3)
Gate: v1.0.0 model with the new IE gate (C1) against v1.0.0	Passes every check, no warning: hand precision 1.000, gap 0
Gate: final_v1 with 4 true matches removed by hand	Fails 4 checks, exit 1: the champion stays
Gate: real IPW run / real 20-trial run	IPW passes (nothing moved); 20-trial run fails, exit 1 (section 4)
Publishing (ci/publish.py)	Unit tests with gcloud and git faked. For real: v1.1.0 published by s11 itself (5 files, annotations mlflow-v2, git-6270fda); it first refused because of an uncommitted file, as designed
Release v1.1.0	The bundle reloads and scores the hand sheet like s06 (max diff 5e-7): precision 1.000, recall 1.000, 0 FP; MLflow matching-same-device v2 @challenger
Scope, secrets, cost	Only Model-Development/matching and model-training.yml changed; no keys; no BigQuery query; experiments ran with --no-mlflow; the release wrote MLflow through the server only (one writer)

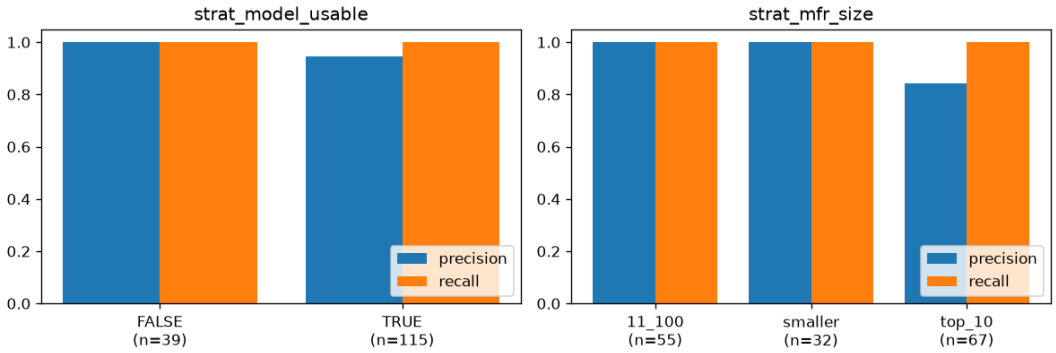
2. Guidelines, item by item

Before = the 12 November check. Now = this branch. **Ready** = the code is in place and tested, but it only runs in CI once the GCP variables are set (section 5).

Guideline	Before	Now	What changed
2.1 Load data from the data pipeline	**Done**	**Done**	Unchanged (s01-s04 from PROD).
2.2 Train several models, select the best by metrics	**Partial**	**Partial**	The gate now decides by metrics whether a new model replaces the champion, and a chart compares all four models. Which of the four variants is released (xgb_no_match_kind) is still chosen by hand in EXPERIMENTS.md.
2.3 Validation on a hold-out set	**Done**	**Done**	Unchanged.
2.4, 2.5, 6 Bias slices, report, mitigation	**Partial**	**Done**	s08 writes each slice with its n. The gate computes the widest precision / recall gap over strat_model_usable and strat_mfr_size (slices with ≥ 8 accepted pairs or matches) and writes bias_report.md and bias_slices.png. Mitigation: IPW weights as sample_weight, tried, no effect (section 4).
2.6, 8.7 Push the model to a registry	**Partial**	**Done**	s11 runs the upload and annotation itself (ci/publish.py); used for v1.1.0. --print-only keeps the old behaviour. The CI run of it is under 7.4.
3 Hyperparameter tuning, documented	**Done**	**Done**	s06 now also saves every trial (trials.csv).
4 Experiment tracking (MLflow)	**Done**	**Done**	Unchanged.
4 Charts comparing the models	**Partial**	**Done**	eval/compare_models.py: one bar chart of the scorecard metrics per model (comparison.png).
5 Sensitivity analysis	**Partial**	**Done**	eval/hparam_sensitivity.py on 60 trials: validation AP against each hyperparameter (section 4).
7.1 CI trains when model code changes	**Partial**	**Ready**	model-dev-integration added to the push branches; still needs the GCP variables.
7.2 CI validation with a threshold	**Missing**	**Done**	ci/gate.py with fixed limits in ci/gate.json (dated 2026-11-13) against ci/champion_metrics.json (now v1.1.0); a failed rule fails the job.
7.3 CI bias check that alerts or blocks	**Missing**	**Done**	Bias gaps are gate ceilings: above the ceiling fails; only warns if the champion already broke it and the new model is no worse (the chatbot's rule).
7.4 Automatic registry push / deployment	**Missing**	**Ready**	Last CI step, push only, after the gate passed: s11 --semver auto --lineage ci.
7.5 Notifications	**Done**	**Done**	GitHub failure e-mails; the gate's checks also go to the run summary.
7.6 Rollback if the new model is worse	**Missing**	**Done**	Gate failure = no publish, champion stays. Shown on a real run (20-trial retrain blocked).
8.1 Docker	**Missing**	**Done**	Dockerfile runs ci/train_chain.sh: s05 → s06 → s08 → comparison → gate.
8.6 Model selection after the bias check	**Missing**	**Done**	The gate includes the bias gaps.

3. The gate

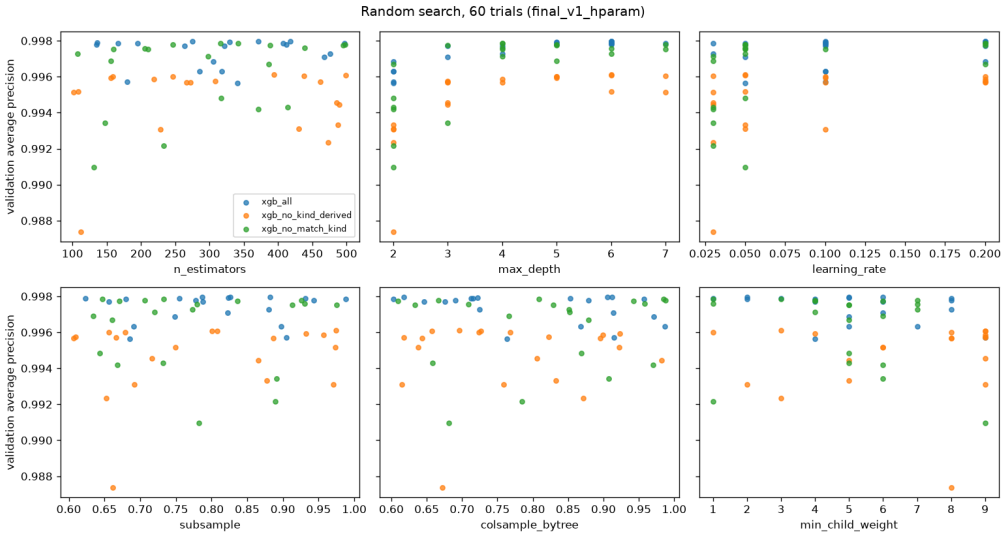
- **Floors** (always fail): hand recall ≥ 0.99, strict hand precision ≥ 0.90, test precision ≥ 0.98.
- **Ceilings**: IE accept rate ≤ 0.35, NON_MATCH accept rate ≤ 0.08, precision gap ≤ 0.15, recall gap ≤ 0.10.
- **Not worse than the champion by more than**: hand precision 0.04 (about 3 false positives, the noise EXPERIMENTS.md reports), hand recall 0.01, test precision 0.005, test recall 0.02, IE accept rate 0.05.
- Limits were fixed on 13 November before any new model was scored. v1.0.0 broke the precision-gap ceiling (top_10 0.842 vs 1.000 = 0.158; a warning). The champion is now **v1.1.0**, whose gap is 0.
- CI cannot reach the local MLflow registry, so the champion's numbers are a committed file, as for the chatbot. A version CI publishes is annotated `mlflow=gh-run-<id>` (the run's artifact, kept 90 days, holds its mlflow.db) and `git=git-<sha7>`.



v1.0.0 precision and recall per slice, from the gate's bias report (n = candidate pairs on the hand sheet). The top_10 gap is the 3 contains false positives; with v1.1.0 every slice is 1.000.

4. Experiment results (13 November)

Run	What	Gate	Result
final_v1_ipw	final_v1 recipe + IPW weights (weight_full_trimmed averaged over event types, joined on tier × year, scaled to mean 1, range 0.64-4.10)	Pass	No effect: identical hand results (0.959, 3 FP, recall 1.000), same slice gaps; test precision / recall move ≤ 0.0004. Dropped; the trade-off is no cost and no gain.
final_v1_hparam	final_v1 recipe with 20 trials (60 in all)	Fail	Hand 0.899 (8 FP, 7 contains) at the same validation AP; NON_MATCH accept 0.096, precision gap 0.333. Blocked: the rollback works on a real run. Only max_depth matters (Spearman 0.65-0.84), then learning_rate.
C1 / final_v1_c1 → v1.1.0	The 3 contains false positives of v1.0.0, fixed with a serving gate	Pass	Cause: 4-digit MAUDE key inside a longer GUDID model, product codes disagree; the label rule id_contains_same_labeler labels all 28 such training pairs MATCH, the hand guide (DEF) says NON_MATCH. Fix: IE gate e6_e3b_c1 sends them to INSUFFICIENT_EVIDENCE (a bundle applies the gate version in its meta.json, so v1.0.0 keeps its decisions). Same model: hand 0.959 → **1.000** (0 FP), recall 1.000, NON_MATCH accepted 3 → 0, slice gap 0.158 → 0. Cost: 5 of 52,436 test-split matches go to IE. Released as v1.1.0. Optimistic: the rule was written after seeing these FPs.



Validation average precision of every random-search trial against each hyperparameter (final_v1_hparam).

5. What is left

Done since the first version of this update

- Branch `model-dev/guidelines-ci` committed and pushed (dfff3dc, 6270fda); open the PR into model-dev-integration. It runs the tests and the smoke job, including the gate in report-only mode.
- The contains fix: serving gate e6_e3b_c1, released as v1.1.0, now the gate's champion.
- Hand batch 2 (E5): taken off the task list. The hand sheet stays the only hand set; it was used to choose and to test, so the hand numbers (1.000 for v1.1.0) are optimistic.

1. GCP setup for CI (with Nainica; ask before creating anything)

- Repository variables GCP_WORKLOAD_IDENTITY_PROVIDER, GCP_SERVICE_ACCOUNT, GCP_PROJECT_ID.
- Service account roles: BigQuery jobUser + dataViewer (as before), Artifact Registry writer on vigil-md-models, and object create/view on gs://vigil-md-raw-nd (s06 copies the training table there). Listed in docs/CI_CD.md.
- Confirm the provider name: docs/CI_CD.md says `vigil-md`, the chatbot uses `vigil-md-repo`.
- Then run it once by hand (Actions → model-training → Run workflow). Keep --max-gb 2.

2. Optional

- Move the MLflow @champion alias to v1.1.0 (today it carries @challenger).
- Fix the label rule itself (C1 option a) in a later label set, so training agrees with the hand guide.
- Rerun the IPW and 20-trial experiments with MLflow on, and copy the charts into reports/ (today they are in the git-ignored data/runs/).

6. How to run the checks above

From Model-Development/matching, no GCP, no cost (these overwrite data/hand_features.csv, data/scores/ and data/cutoffs.json):

```
pip install -r requirements.txt -r requirements-ml.txt
python -m pytest -q -rs
python ci/make_ci_feature_table.py --out data/ci/features.csv --feature-set fs2 \
    --tfidf data/ci/features.tfidf.pkl
python scripts/s05_build_hand_features.py --feature-set fs2 --tfidf data/ci/features.tfidf.pkl
python scripts/s06_train_models.py --data data/ci/features.csv --trials 1 --no-mlflow
python ci/check_training_outputs.py --title Smoke
python eval/s08_all.py --exp none --ie-gate --hand-features data/hand_features.csv --no-mlflow
python ci/gate.py --report-only
docker build -t vigil-matching-train .   # then see the Dockerfile header to run it
# gate an experiment: python ci/gate.py --scores-dir data/runs/<exp>/scores
```